SCF-PDP Programming Exercise Report

Heuristic Optimization Techniques, WS 2025

Daniel Levin (12433760) Kartik Arya (12402369)

1 General Setup

1.1 Implementation Environment

The implementation is developed in Python 3.13. The following external libraries are utilized:

- `numpy` – efficient array operations and numerical computations
- `scipy` – pairwise matrix computation of Euclidian distances via `cdist`
- `pymhlib` – metaheuristic framework providing the base `Solution` and `SA` classes

1.2 Core Classes

The implementation consists of the following main classes:

`SCFPDPInstance` Parses and stores problem instance data including the number of requests n, vehicles n_K, capacity C, minimum requests γ, and fairness weight ρ. Precomputes the distance matrix using vectorized Euclidean distance calculations.

`SCFPDPSolution` Extends `pymhlib.Solution`. Represents a solution as a list of `Route` objects. Provides objective calculation combining total travel distance with the Jain fairness measure, and solution validation. It supports delta evaluation of the objective value, if the corresponding flag is passed at initialization.

`Route` Represents a single vehicle route as an ordered sequence of location indices. Tracks served requests, route distance, and enforces capacity constraints. Supports request insertion with automatic pickup-before-dropoff ordering.

1.3 Experimental Setup

1.3.1 Implementation Stack

- `pandas` – tabular result aggregation and formatting
- `matplotlib` – visualization and plot generation

1.3.2 Comparison Framework Architecture

A modular algorithm comparison framework was developed to enable systematic evaluation of different heuristics. The framework is built around the following core data structures:

`AlgorithmConfig` Encapsulates an algorithm's identity: a human-readable name, the algorithm class, and initialization parameters. This abstraction allows any algorithm conforming to the construction heuristic interface to be plugged into the comparison pipeline.

`InstanceResult` Captures per-instance metrics: objective value, construction time, total runtime (including instance parsing), and instance metadata.

`ComparisonResults` Aggregates pairwise comparison statistics for a given instance size: mean and standard deviation of objectives for both algorithms, percentage difference, and win/loss/tie counts.

`ExperimentSummary` Aggregates results across all instance sizes, providing total win counts and enabling cross-size analysis.

1.3.3 Pairwise Comparison Methodology

The comparison proceeds as follows:

1. Both algorithms are executed on each instance in the test set.
2. For each instance, the algorithm achieving the lower objective value is recorded as the "winner."
3. Statistics are computed per instance size: mean objective $\pm$ standard deviation, and percentage difference defined as
$$\frac{obj(\text{Alg}_1) - obj(\text{Alg}_2)}{obj(\text{Alg}_1)} \times 100$$
4. Results are aggregated into summary tables and visualizations.

For randomized algorithms, multiple independent runs are performed and averaged to reduce statistical variance.

1.3.4 Extensibility

The framework can be extended beyond pairwise comparison:

- **Multi-algorithm comparison**: By iterating over all pairs of algorithms, a full comparison matrix can be constructed. The bar plots described in 1.3.5 can easily be extended by adding more algorithms into the plot of win distribution.
- **Within-algorithm comparison**: The same infrastructure supports comparing an algorithm with different parameters to itself (e.g., RCL size k) by treating each parameter configuration as a distinct algorithm.

1.3.5 Visualization Infrastructure

A unified plotting module (`utils.py`) provides configurable two-panel figures via the `ExperimentPlotConfig` dataclass. Standard visualizations include:

- Objective value vs. instance size (line or bar plots)
- Runtime vs. instance size
- Percentage difference and win distribution across instance sizes

Plots support logarithmic scales, multiple series overlay, and automatic file export.

1.3.6 Hardware

All of the experiments were run on our local machines - MAC with M1 Pro Chip, 8 cores and 16Gb. For algorithms comparison multiprocessing was used to expedite the total runtime, for larger instances of the problem number of concurrent processes was limited to 2 to avoid memory explosion caused by replicating the instance related classes over multiple processes.

1.3.7 Runtime limits

With the multiprocessing mentioned above, the runtime of the algorithms that converge in finite time (no `while true`) never exceeded 5 minutes, so we didn't need to impose an artificial timeout. For algorithms such as SA - the runtime is controlled by algorithm-specific parameters that were adjusted for larger instances.

2 Deterministic Construction Heuristic

2.1 Idea

The greedy construction heuristic builds a feasible solution by iteratively selecting the closest unserved request to the current insertion point on each route. The routes are iterated sequentially and each route gets exactly one request per iteration. Requests are added one at a time, with pickup and dropoff locations inserted adjacently at the end of the route (before returning to depot), ensuring capacity constraints are satisfied at every step. This is a very primitive greedy heuristic that served as the baseline. The benefit of this approach is that it distributes the requests between vehicles evenly. However, as we experimented with the provided instances we noticed that the ρ value is usually low so the impact of the `Jain` value on the final objective function is minor compared to the sum of distances.
The algorithm guarantees feasibility if a solution exists, raising a `ValueError` otherwise.

2.2 Algorithm

The algorithm maintains a set of served requests and iterates over all vehicle routes until the minimum required requests (γ) are fulfilled. For each route:

1. Select the insertion position (end of route)
2. Find the closest feasible request using Euclidean distance
3. Check capacity constraint at insertion point
4. If no feasible request exists, mark route as full and continue to next vehicle
5. Insert pickup and dropoff locations consecutively (dropoff distance = 1)
6. Repeat until γ requests are served or all routes are marked as full

The selection process iteratively excludes non-fitting requests and searches for the next closest alternative until a feasible request is found.

2.3 Challenges

When we started running the greedy construction heuristic on larger instances, we noticed that the runtime increased significantly. We first applied manual timing to the flow to pinpoint the bottleneck in the instance initialization, after which we used the `line_profiler` tool to profile the function by line. We discovered that the distance matrix calculation implemented in pure Python is very inefficient, resulting in approximately 20 seconds of runtime for an instance of size 2000. After applying NumPy vectorized operations for calculating distances using the underlying CPython, which replaced the pure Python loops with a manual Euclidean distance calculation, the runtime dropped by a factor of 100 (e.g., 0.2s for an instance of size 2000). Since the largest instances provided are of size 10000, the parsing runtime for them still remained relatively high—approximately 20 seconds. To address this problem, given that we will be required to run many experiments on main instances, we integrated `scipy.cdist` function, which optimizes Euclidean distance calculation and effectively reduces the parsing runtime for instances of size 10000 from 20 seconds to 2-3 seconds. See Figure 1 for visualization of the runtimes of instances parsing and greedy construction itself.

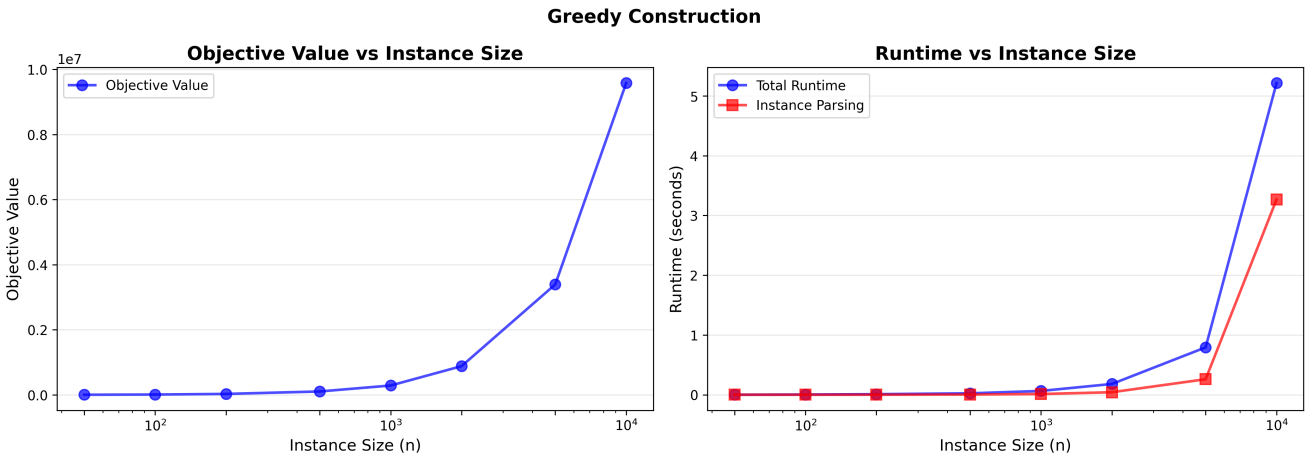

Figure 1: Instances Parsing and Greedy Construction

2.4 Experiments

Most of the experiments with greedy construction evolved around optimizing the instance parsing (see subsection 2.3) and modeling the data structures (see subsection 1.2) representing evolving solutions. This work served as a baseline for all other algorithms later on. We have decided to stick to the simplest approach we came up with initially for multiple reasons:

- **Simplicity** - in the beginning it was important to make sure all the classes, data structures and solution validations behave correctly and for that it was beneficial to test the simplest construction heuristic that can quite easily be cross-checked by a human.
- **Neighborhoods potential** - in general we wanted to find out how much exploration can different neighborhoods bring and for that it was more interesting to start with more primitive solutions instead of building promising solution by simple construction heuristic.
- **Time constraints** - due to the time constraints of the assignment and the number of algorithms and instances we wanted to experiment with throughout this exercise, we decided to save time on developing smarter greedy heuristics.

3 Randomized Construction Heuristic

3.1 Idea

The randomized construction heuristic extends the deterministic greedy approach by introducing controlled randomness into the request selection process. Instead of always selecting the single closest feasible request, the algorithm maintains a *Restricted Candidate List* (RCL) of closest feasible candidates and randomly selects among them. Feasible candidates in this case means that by adding these candidates to the chosen position on the route, the capacity constraint is not violated.

This randomization serves two purposes:

1. **Solution diversity**: Different executions produce different solutions, enabling exploration of the solution space when applied iteratively (e.g., in GRASP).
2. **Escaping greedy traps**: The purely greedy approach may lead to suboptimal solutions by making locally optimal choices that are globally poor. Randomization allows the algorithm to occasionally select non-optimal immediate choices that may lead to better overall solutions.

The construction process remains identical to the greedy heuristic: requests are assigned to vehicles in a round-robin fashion, with pickup and dropoff locations inserted at the end of each route. The only modification is the request selection mechanism.

3.2 Randomization Control (RCL)

The degree of randomization is controlled through a *cardinality-based* Restricted Candidate List. At each insertion step, the RCL is constructed as follows:

1. Compute distances from the current insertion position to all unserved pickup locations.
2. Filter out requests that would violate the vehicle capacity constraint.
3. Sort remaining candidates by distance in ascending order.
4. Select the top k closest candidates to form the RCL, where k is a tunable parameter (`top_random_pickups_to_consider`, fine-tuned default $k = 10$ - see Fig. 2).
5. Uniformly at random select one request from the RCL.

The parameter k controls the trade-off between greediness and randomness:

- $k = 1$: Equivalent to the deterministic greedy heuristic.
- Small k: Biased towards high-quality greedy choices with limited diversity.
- Large k: Greater diversity but potentially lower average solution quality.

For choosing the optimal value of `top_random_pickups_to_consider` we ran the experiments on 4 different instances with multiple `top_random_pickups_to_consider` values each, and for each of the runs we calculated the average objective value and runtime over 10 constructions. Based on the results in Figures 2,3,4,5 we have set the value of `top_random_pickups_to_consider` to 10 which showed stable results for every instance size. However, it is important to note that there was no consistency in objective value as the RCL size grows. This can be explained by "primitivity" of the greedy choice that the randomized version heavily relies on. Each route is a standalone TSP problem once requests for that route are fixed and a heuristic that always inserts requests in the end of the route cannot achieve good improvement in general.

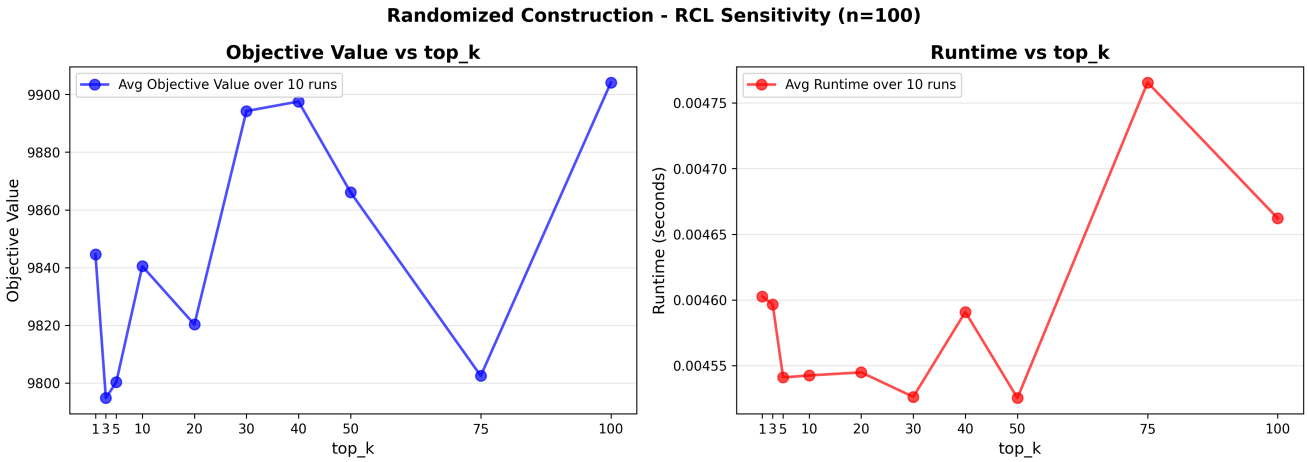

Figure 2: Random construction (n=100)

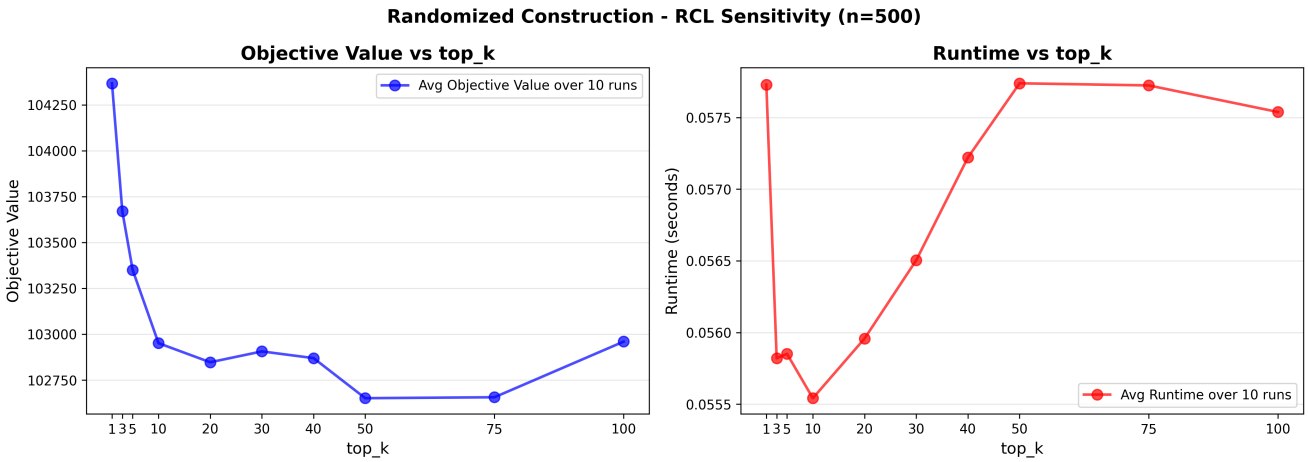

Figure 3: Random construction (n=500)

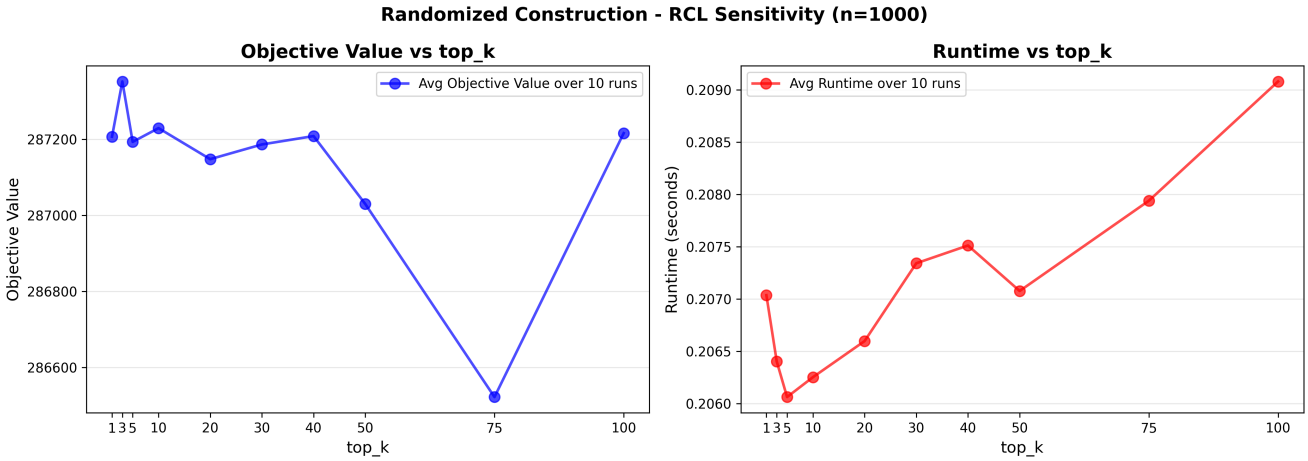

Figure 4: Random construction (n=1000)

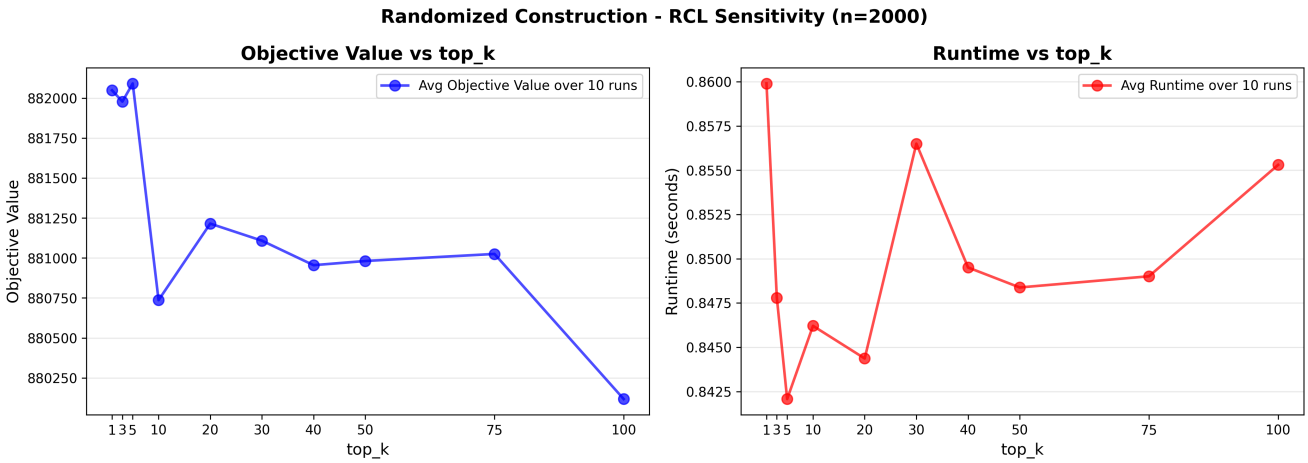

Figure 5: Random construction (n=2000)

3.3 Comparison with deterministic

To compare two algorithms we built the framework described in 1.3.3 - see Fig. 6. The data that these plots are based on is summarized in the table 1. Every instance under the `test` folders was used for

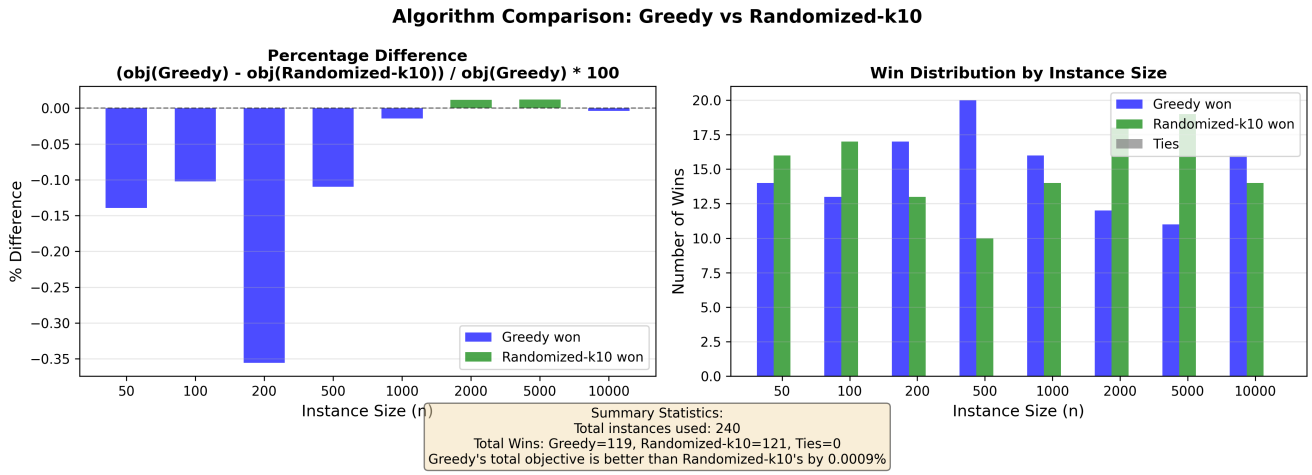

Figure 6: Greedy vs Randomized Construction

this comparison and average values were aggregated per instance size (8 average objective values per algorithm each representing 30 solution). It can be seen from both the table and the plots that on most of the instance sizes, greedy construction performs better in terms of total objective value, however, out of 240 instances it absolutely won 119, which is slightly less than half. Another interesting observation is that even when overall average of the instances of a certain size is better (=lower) for one algorithm, the number of absolute wins for that instance size of this same algorithm might be lower (see Fig. 6). This can be explained by non-linear impact of single decisions of this construction approach - one wrong decision of randomized algorithm might lead to a larger gain of the objective value. This analysis together with some experimenting we've conducted on randomized construction heuristic has brought us to the conclusion that the solutions it generates are not very diverse. The reason for that is the primitivity of the greedy construction approach randomized approach is built upon, that doesn't allow to generate more diverse routes for example with dropoffs not following the corresponding pickups strictly. However, we decided to stick to the current approach for reasons such as simplicity and time limit.

Table 1: Greedy vs Randomized-k10 performance

Size	Greedy Obj	Randomized-k10 Obj	% Diff	Wins (G/R/T)
50	3399.9 ± 255.6	3404.0 ± 252.1	-0.14 ± 1.66	14/16/0
100	9550.4 ± 526.4	9558.4 ± 511.2	-0.10 ± 1.46	13/17/0
200	26951.1 ± 1478.9	27044.7 ± 1453.9	-0.36 ± 0.84	17/13/0
500	107992.8 ± 3909.8	108113.3 ± 4004.6	-0.11 ± 0.43	20/10/0
1000	301485.7 ± 8934.4	301531.2 ± 9020.0	-0.01 ± 0.24	16/14/0
2000	865744.6 ± 35617.7	865629.7 ± 35261.4	+0.01 ± 0.14	12/18/0
5000	3446920.2 ± 141250.3	3446515.4 ± 141514.6	+0.01 ± 0.06	11/19/0
10000	9712754.5 ± 292443.2	9713139.8 ± 292974.5	-0.00 ± 0.03	16/14/0
TOTAL	434243975.3	434248090.3		119/121/0

4 Beam Search Heuristic

4.1 Idea

Beam search utilizes the functionality of randomized construction heuristic by maintaining multiple partial solutions (beam) during construction. The algorithm uses two key parameters: **beam width** (β) and **branching factor** (bf). Branching factor determines how many closest pickups will be conisdered at each level of the beam search. At each construction step, β best partial solutions are kept in memory. For each partial solution, successor solutions are generated using the heuristic of the construction heuristics:

1. Randomly selecting a route for extension (this has shown better results than for example choosing the route with minimal distance, although the final solution is usually less fair)
2. Using the RCL mechanism used by randomized construction to identify a pool of closest candidate requests to branch on
3. Creating new partial solutions by inserting different requests from the RCL

The branching factor controls exploration breadth at each node, while beam width limits the total number of parallel solution paths considered at each step. Both insertion positions and dropoff distances are selected using the randomized construction heuristic's RCL-based selection. The algorithm terminates when all solutions in the beam are complete, returning the best solution found.

4.2 Experiments

Parameter sensitivity analysis was conducted to determine optimal values for beam width and branching factor. The following parameter ranges were tested:

- **Beam width** (β): {2, 3, 5, 7, 10, 15, 20, 25, 30}
- **Branching factor** (bf): {2, 3, 4, 5, 6, 8, 10}

Experiments were performed on instances of sizes $n \in \{100, 200, 500\}$, evaluating all 63 parameter combinations per instance. Results are visualized as heatmaps showing objective values and runtimes across parameter space, with optimal combinations marked (see Fig. 7 8 9). The experiments reveal the trade-off between solution quality (improved with higher β and bf) and computational cost (increased quadratically with both parameters). Best configurations vary by instance size, with smaller instances benefiting from higher parameter values while larger instances require lower values to remain within time constraints. Interestingly, the branching factor seems to have more clear impact on the solution quality than beam width. As seen in Fig. 7 for example, the largest beam width doesn't guarantee best objective value, while largest branching factor always results in best objective value across the examined configurations. This can be explained by the randomness of underlying randomized construction and RCL slicing performed by the beam algorithm. Larger branching factor allows the algorithm to look into more diverse solutions thanks to larger RCL that it generates. In this case even lower beam width are able to discover promising solutions at every level of the tree. The runtime is quadratically proportional to the beam width and branching factor which can be clearly seen in the heatmaps on the right side of figures 7 8 9. As a general note, it is clear that the beam search is not very powerful - the improvement of the objective value is not significant which can be explained by weak exploration ability of beam search and underlying construction heuristic.

After inspecting the heatmaps, we have decided to choose $\beta = 5$ and $bf = 10$ as our final parameters, although the results might vary depending on the instance.

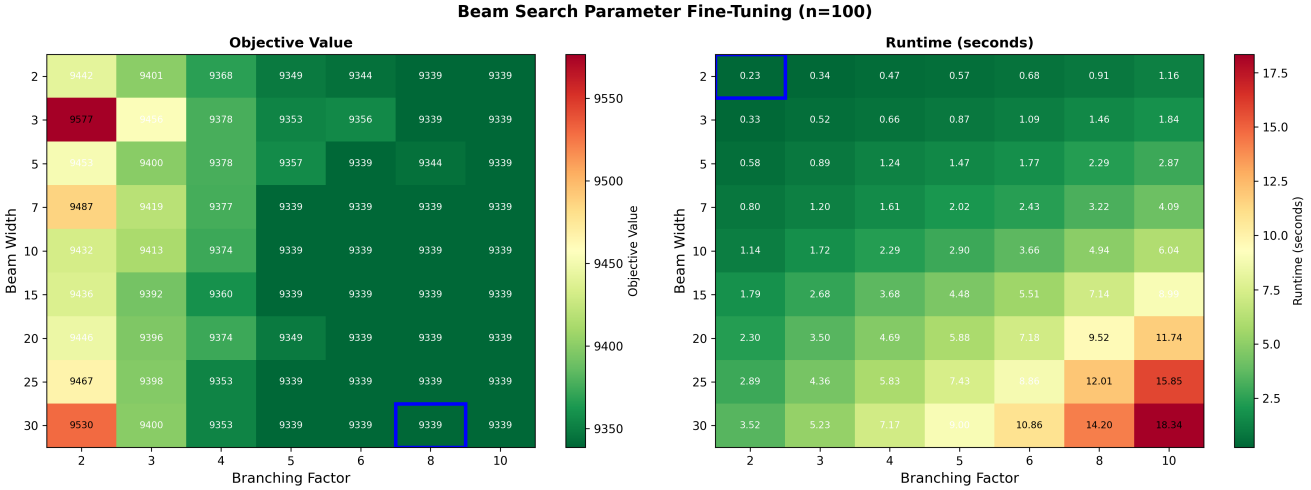

Figure 7: Beam Search (n=100)

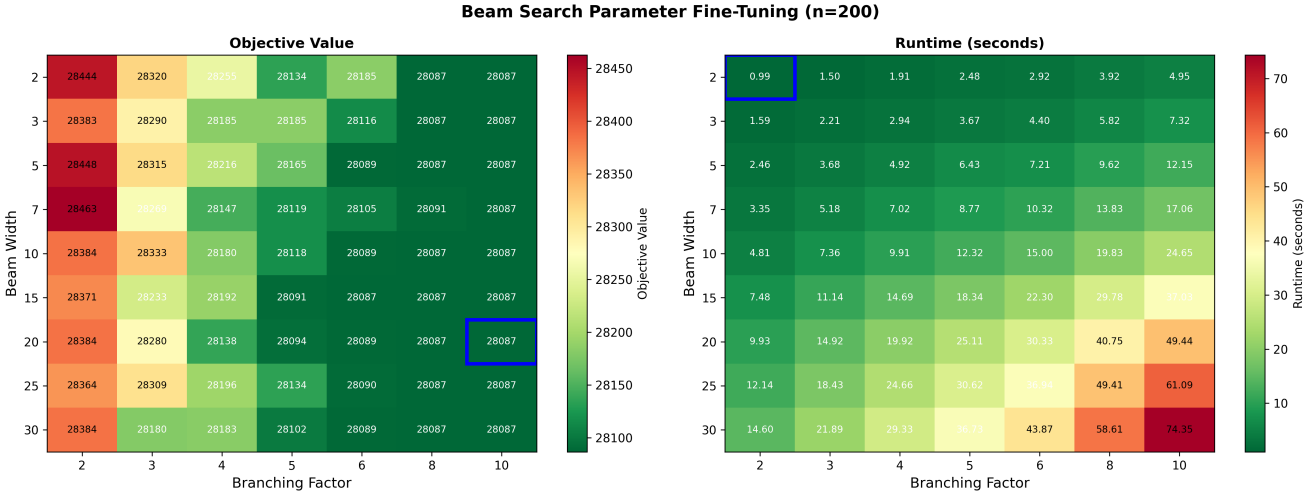

Figure 8: Beam Search (n=200)

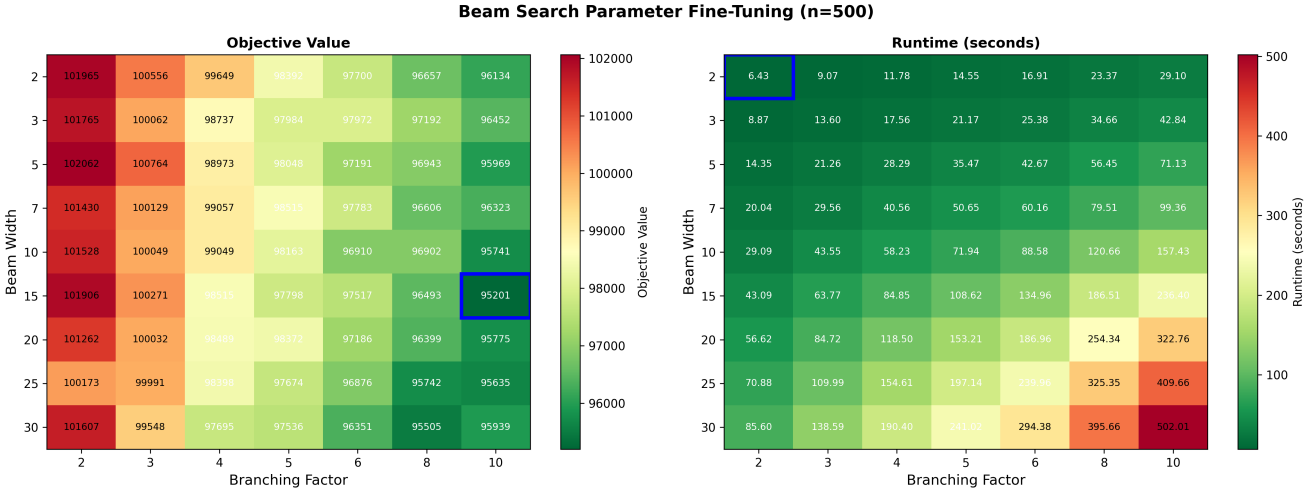

Figure 9: Beam Search (n=500)

5 Local Search Framework

5.1 General Framework

The local search algorithm operates on a current solution consisting of routes R_k for all $k \in K$. It repeatedly explores a sequence of neighborhoods $\{\mathcal{N}_1, \ldots, \mathcal{N}_m\}$ using a VND-style sweep. Starting from an initial feasible solution, each neighborhood $\mathcal{N}_j$ is processed in a deterministic order. At each neighborhood:

1. Apply the step strategy (*first-improvement* or *best-improvement*) via

$$(\text{candidate}, \text{improved}) = \text{step\_strategy.improve}(\text{current}, \mathcal{N}_j)$$

2. If an improving move is found (improved = True), update the current solution to candidate and restart the sweep from the first neighborhood.

3. Repeat until no neighborhood yields improvement or a time limit is reached.

This design ensures feasibility at all times (respecting vehicle capacities and single-vehicle service per request) and leverages delta-evaluation inside neighborhoods for efficiency. The repeated restart after each improvement effectively implements a VND-style search.

5.2 Step Functions

Two step strategies determine how neighbors are accepted during the local search:

1. **First-Improvement:** Neighbors are generated lazily, and the first improving neighbor (with lower objective) is immediately accepted. This approach reduces iteration cost and quickly navigates plateaus in large neighborhoods.

2. **Best-Improvement:** All neighbors in the neighborhood are evaluated, and the one with the best objective reduction is selected. Although more computationally intensive per iteration, this strategy provides a more stable convergence towards local optima by systematically selecting the most beneficial move.

Both strategies are implemented through a generic interface `StepStrategy`, and can be seamlessly integrated with any neighborhood class that implements `generate_neighbors()`. The LocalSearch class encapsulates repeated sweeps, while the VND wrapper orchestrates multiple neighborhoods in sequence.

6 Neighborhood Structures

We implement three neighborhoods that operate directly on the route representation in SCF-PDP. All use lazy generation and delta-evaluation to efficiently produce feasible neighbors.

- **Insert Neighborhood.** Moves a single pickup or dropoff within the same route. For route k, a location at pos_{from} is reinserted at pos_{to} using

`route.move_from_to(pos_from, pos_to).`

  All valid pairs of positions are explored, generating neighbors that preserve request feasibility and capacity. This neighborhood is mainly for fine-grained reordering of stops.

- **Swap Neighborhood.** Exchanges two stops i and j within the same route via

`route.swap_locations(i, j).`

  It enumerates all pairs $i < j$, producing neighbors that improve route sequencing and fairness without changing which requests are served.

- **Relocate Neighborhood.** Moves a full request (pickup + dropoff) to a different position in the same route or another route using

  ```
  route_from.relocate_request_to(request_id, route_to, pos_to, 1).
  ```

  Iterating over all requests and possible target positions, this neighborhood redistributes workload across vehicles and adjusts fairness, enabling larger structural improvements than Insert or Swap.

7 VND Framework

The Variable Neighborhood Descent (VND) applies a sequence of neighborhoods to iteratively improve a solution. By default, the neighborhoods are ordered as **Insert**, **Swap**, and **Relocate**, but this order can be modified if needed.

7.1 Neighborhood Order

VND cycles through the neighborhood list in order. Whenever an improvement is found in a neighborhood, the search immediately restarts from the first neighborhood. This ensures that early neighborhoods are fully exploited and prevents missing improvements in higher-priority moves.

7.2 Impact of Order

The fixed order affects convergence: placing neighborhoods that generate smaller, frequent improvements first (like Insert) allows rapid local adjustments, while larger structural moves (like Relocate) are considered later. Changing the order can alter the final local optimum and the efficiency of search.

7.3 Stopping Criterion

VND terminates when a complete pass over all neighborhoods fails to produce any improvement, or an optional time limit is reached. This guarantees a local optimum with respect to the entire set of neighborhoods under the selected step strategy (*first-improvement* or *best-improvement*).

8 GRASP

The Greedy Randomized Adaptive Search Procedure (GRASP) iteratively constructs solutions using a randomized heuristic and improves them via local search. The procedure consists of three main components:

8.1 Construction Phase

A randomized construction heuristic builds an initial feasible solution. While GRASP defines a parameter $\alpha \in [0, 1]$ to control greediness versus randomness, in the current implementation the heuristic selects requests randomly without actively using α. In general, $\alpha = 0$ would yield purely greedy selection, and $\alpha = 1$ fully random selection.

8.2 Local Search Phase

Each constructed solution is improved using a VND over the three neighborhoods: **Insert**, **Swap**, and **Relocate**. The step strategy is *First-Improvement*, though *Best-Improvement* can be used as an alternative. The VND iterates through neighborhoods until no further improvement is possible.

8.3 Iteration and Selection

GRASP repeats the construction and local search phases for a maximum of `max_iter` iterations. After each iteration, the solution is evaluated, and the one with the best objective value is retained. This combination of randomized construction and systematic local search allows GRASP to explore diverse solutions while converging to high-quality local optima.

9 Advanced Metaheuristic (Simulated Annealing)

9.1 Idea

Simulated Annealing is implemented using the `pymhlib` framework with problem-specific adaptations for SCF-PDP. The algorithm can be initialized using three construction methods: (1) beam search constructor with $\beta = 10$ and $bf = 4$, (2) randomized construction heuristic, or (3) greedy construction heuristic.

The neighborhood exploration uses one of three problem-specific neighborhoods:

- **Insert:** Move a pickup or dropoff to a different position within the same route
- **Swap:** Exchange two locations within the same route
- **Relocate:** Move an entire request (pickup + dropoff) to a different route

At each iteration, a random neighbor is generated from the selected neighborhood. The move is evaluated using delta evaluation (see Section on Delta Evaluation) to efficiently compute the objective change $\Delta = \text{obj}_{\text{new}} - \text{obj}_{\text{current}}$. Worse moves are accepted with probability $e^{-\Delta/T}$ where T is the current temperature.

Key parameters include: initial temperature $T_{\text{init}}(100)$, geometric cooling factor $\alpha(0.95)$ (applied as $T \leftarrow \alpha \cdot T$), and equilibrium iterations (1000) (number of iterations at each temperature level). The algorithm terminates when the maximum iteration count is reached (10000) or temperature becomes negligible.

9.2 Parameter Tuning

Parameter tuning for Simulated Annealing was performed on training instances of sizes $n \in \{50, 100, 200\}$ using the Relocate neighborhood with delta evaluation enabled. All 3 construction algorithms that have been developed were used for SA initialization via `meths_ch` interface of the constructor. After experimenting with these constructions it was discovered that SA with Beam achieves the best objective value, albeit slower.

Each configuration was evaluated across random subsample of instances in each size category of size 3, and performance was measured by mean objective value, standard deviation, and runtime. The following parameters were tuned:

- **Cooling rate (α) - see Fig. 10:** Controls the geometric cooling schedule $T \leftarrow \alpha \cdot T$. Three values were tested: $\alpha \in \{0.90, 0.95, 0.99\}$. Higher values result in slower cooling, allowing more exploration but potentially increasing runtime.
  - Best configuration: $\alpha = 0.90$
  - Impact: Faster cooling ($\alpha = 0.90$) for some reason converged slightly slower but to better solutions. ($\alpha = 0.99$) achieved worse solution quality at the cost of longer runtime compared to $\alpha = 0.95$ and $\alpha = 0.90$. These results are counterintuitive to the SA essence because theoretically lower α values should result in worse solutions because the exploration is more limited - less temperatures are explored. Also in terms of the runtime - lower values should result in lower runtime because the geometric progression converges faster.

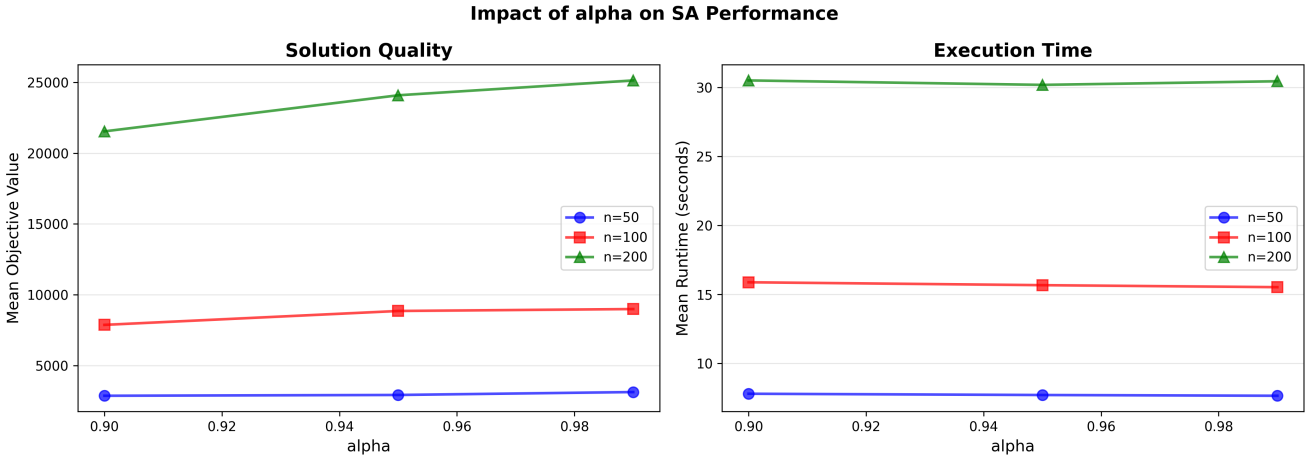

Figure 10: SA Alpha Fine-Tuning

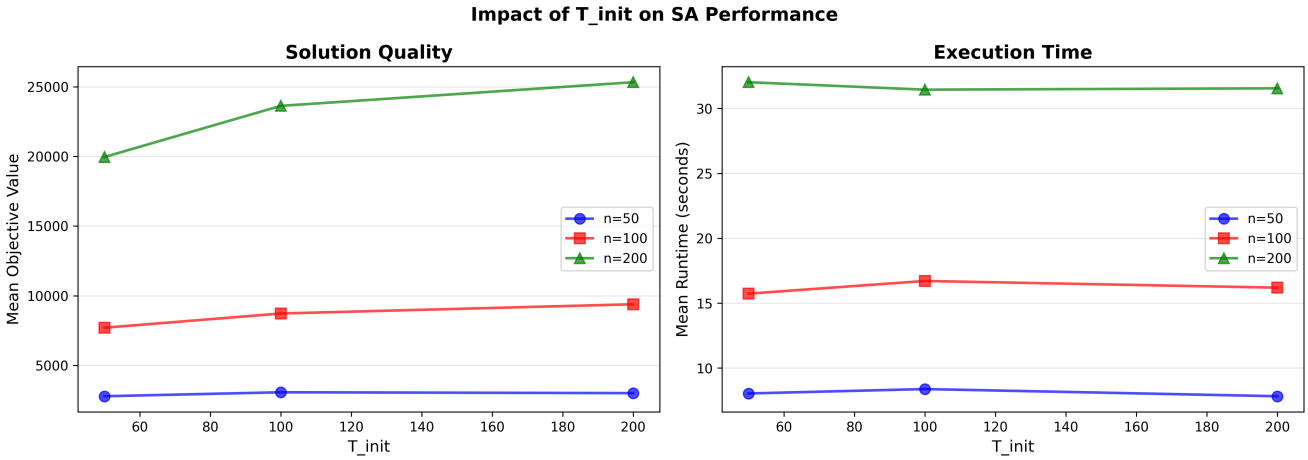

Figure 11: SA Initial Temperature Fine-Tuning

- **Initial temperature (T_{init}) - see Fig. 11:** Determines the initial acceptance probability for worsening moves. Three values were tested: $T_{\text{init}} \in \{50.0, 100.0, 200.0\}$. Higher initial temperatures allow more exploration early in the search.
  - Best configuration: $T_{\text{init}} = 50$
  - Impact: Low temperature ($T_{\text{init}} = 50$) behaved "greedily", which for some reason resulted in better objectives. High temperature ($T_{\text{init}} = 200$) was supposed to allow better exploration with better objective values but ended up being the worst value out of the three. The runtime didn't change significantly for different values of the T_{init}. This is probably due to the fact that small instances were chosen for the evaluation.
- **Equilibrium iterations (equi_iter) - see Fig. 12:** Number of iterations performed at each temperature level before cooling. Three values were tested: equi_iter $\in \{500, 1000, 2000\}$. More iterations allow better exploration at each temperature.
  - Best configuration: $equi\_iter = 500$
  - Impact: Fewer iterations ($equi\_iter = 500$) resulted in better solution quality with slightly slower runtime. More iterations ($equi\_iter = 2000$) didn't provide improvement, neither in solution quality, nor in runtime.

All parameter tuning experiments used a fixed iteration limit of 10000 iterations and the Relocate neighborhood. Final parameter configuration selected: $\alpha = 0.90$, $T_{\text{init}} = 50$, $equi\_iter = 500$.

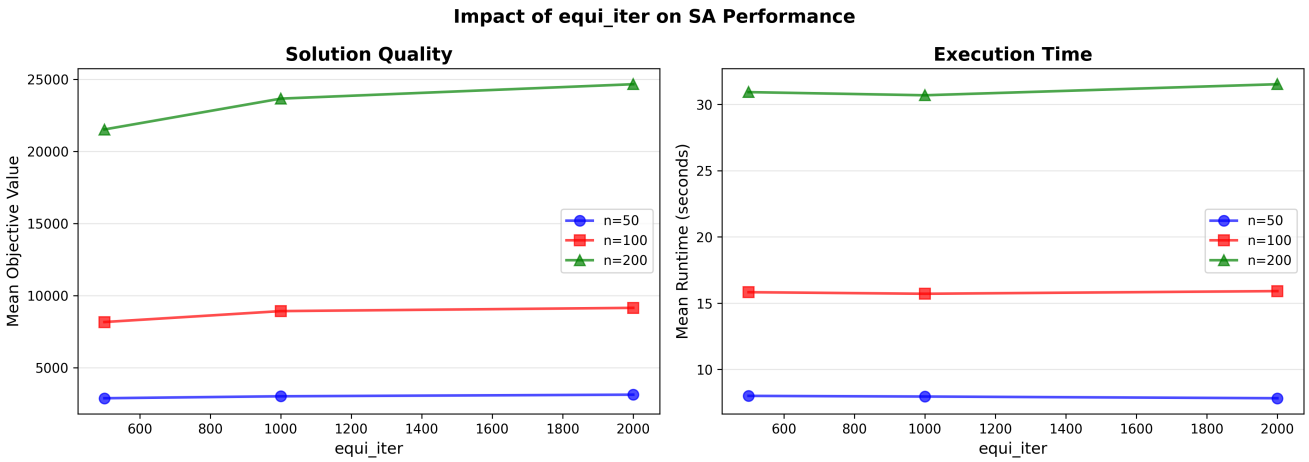

Figure 12: SA Equilibrium Condition Fine-Tuning

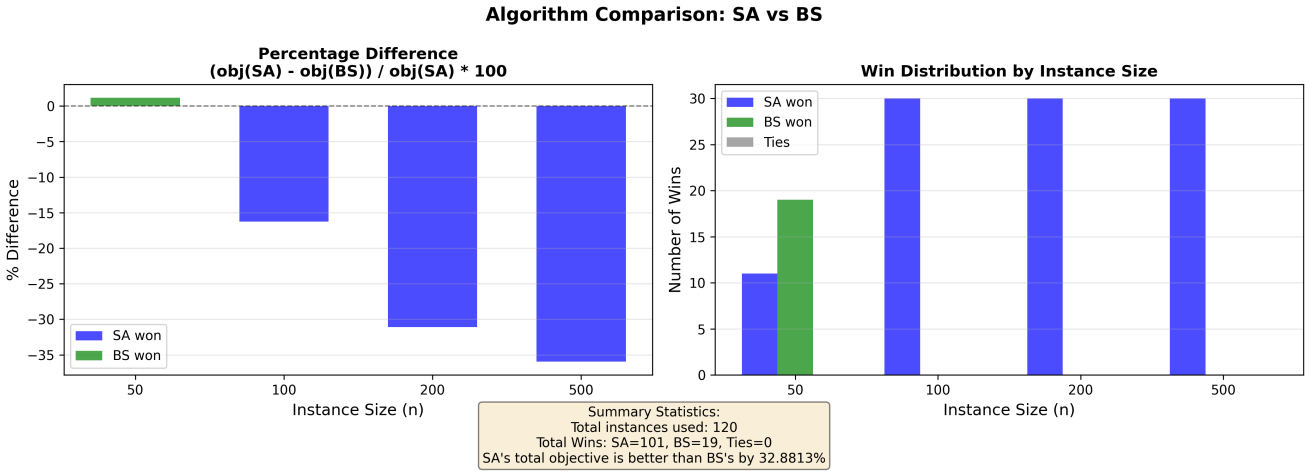

Figure 13: Simulated Annealing vs Beam Search

9.3 Comparison of SA with Beam Search

After experimenting with SA parameters the results felt off so we decided to compare this algorithm with another relatively strong algorithm to make sure it works correctly and it was clearly proved (especially on larger instances) as can be seen in Fig. 13. SA outperformed Beam Search on every instance of size 100 and more with significant improvement in average objective value up to 35% for instances of size 500.

10 Delta Evaluation

10.1 Idea

Delta evaluation is implemented in the `SCFPDPSolution` class to avoid redundant objective function computations during local search. The implementation maintains two cached values: total distance ($\sum d(R_k)$) and sum of squared distances ($\sum d(R_k)^2$), which are incrementally updated whenever a route is modified. The responsibilities distribution between `Route` and `SCFPDPSolution` classes is leveraged to simplify the flow of delta evaluation - `Route` is responsible for incrementally tracking distance of the route (similarly to TSP) while `SCFPDPSolution` only tracks total sum of distances and sum of squares for calculating the final objective value in $O(1)$.

The delta evaluation mechanism uses a callback-based architecture. Each `Route` object holds a reference to the solution's `delta_evaluation` callback method, which is invoked in `recompute_route_distance` whenever a route's changed. The callback receives both the new and old distance values, allowing

incremental updates:

- cached_total_distance ← cached_total_distance + $(new\_distance - old\_distance)$
- cached_sum_of_squares ← cached_sum_of_squares + $(new\_distance^2 - old\_distance^2)$

Delta evaluation is employed by the method `recompute_route_distance` of the `Route` class whenever a change on the route is performed. This allows transparent continuous delta evaluation without the need to track the validity of the current objective value and related variables.

10.2 Performance Analysis

The objective function evaluation without delta evaluation requires iterating over all K vehicles to compute the objective value and for each vehicle all locations of the route are calculated extensively. Even though the distance matrix is pre-calculated, the repetitive access to that matrix might be expensive, especially for larger instances where the matrix won't be able to get cached properly.

- **Without delta evaluation:** $O(K)$ per objective computation (iterate over all routes to sum distances and squares)
- **With delta evaluation:** $O(1)$ per objective computation (direct access to cached values)

The performance improvement is most significant in local search contexts where objective evaluation dominates runtime:

- Neighborhood size for swap/relocate operations: $O(n^2)$ where n is the number of requests
- `BestImprovement` evaluates *all* neighbors: $O(n^2)$ objective calls per iteration
- **Total complexity per local search iteration:**
  - Without delta: $O(n^2 \cdot K)$
  - With delta: $O(n^2)$
  - **Speedup factor:** K (number of vehicles)

10.3 Preprocessing

The implementation leverages preprocessing through the precomputed distance matrix stored in the `SCFPDPInstance` class. All pairwise Euclidean distances $d_{ij} = \lceil \sqrt{(x_i - x_j)^2 + (y_i - y_j)^2} \rceil$ are computed once during instance initialization and stored in $O(n^2)$ space. This preprocessing eliminates repeated square root, rounding operations and matrix access during route distance calculations, reducing the cost of `recompute_route_distance`.

10.4 Experiments

For assessing the impact of the delta evaluation on runtime, we have chosen beam search algorithm with branching factor of 5 and $\beta = 3$ to have the algorithm explore 5 different variations at every iteration. With this setup we have achieved a good balance between number of recalculations triggered during the solution construction and overall runtime of the experiment. The Fig. 14 displays the relative speedup of the delta evaluation for normalized visualization of the results per instance size over 3 randomly sampled training instances of each size. On the left subplot we have measured the impact of branching factor on runtime with and without delta evaluation as branching factor is the main parameter in beam search that affects how many candidates are generated at each step and therefore how many times reevaluation is triggered. Both branching factor and instance size show the effect that delta evaluation has on the runtime as the instance grows. The speedup factor would be even higher if we used an algorithm that employs higher number of neighborhood moves and resulting objective recalculation. Another optimization that

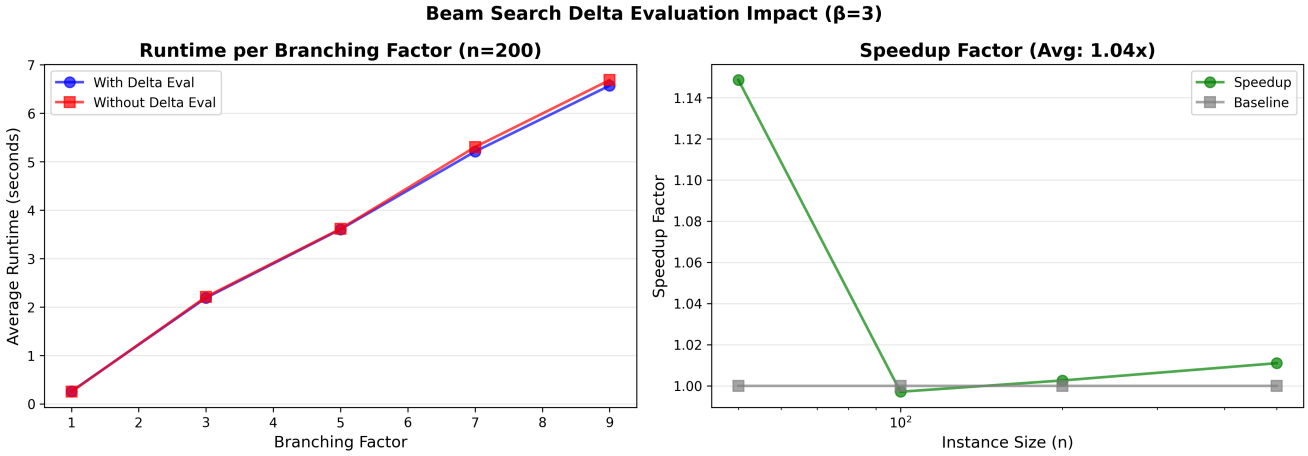

Figure 14: Delta Evaluation with Beam Search

can be introduced to our solution - is lazy objective evaluation as it is designed in the `pymhlib`. For simplicity reasons we decided to implement eager recalculation at every route change assuming that the objective value is looked up after every step.